

Infosys Springboard Virtual Internship

7.0

AI Career Companion Agent for Internship Matching and Interview Preparation

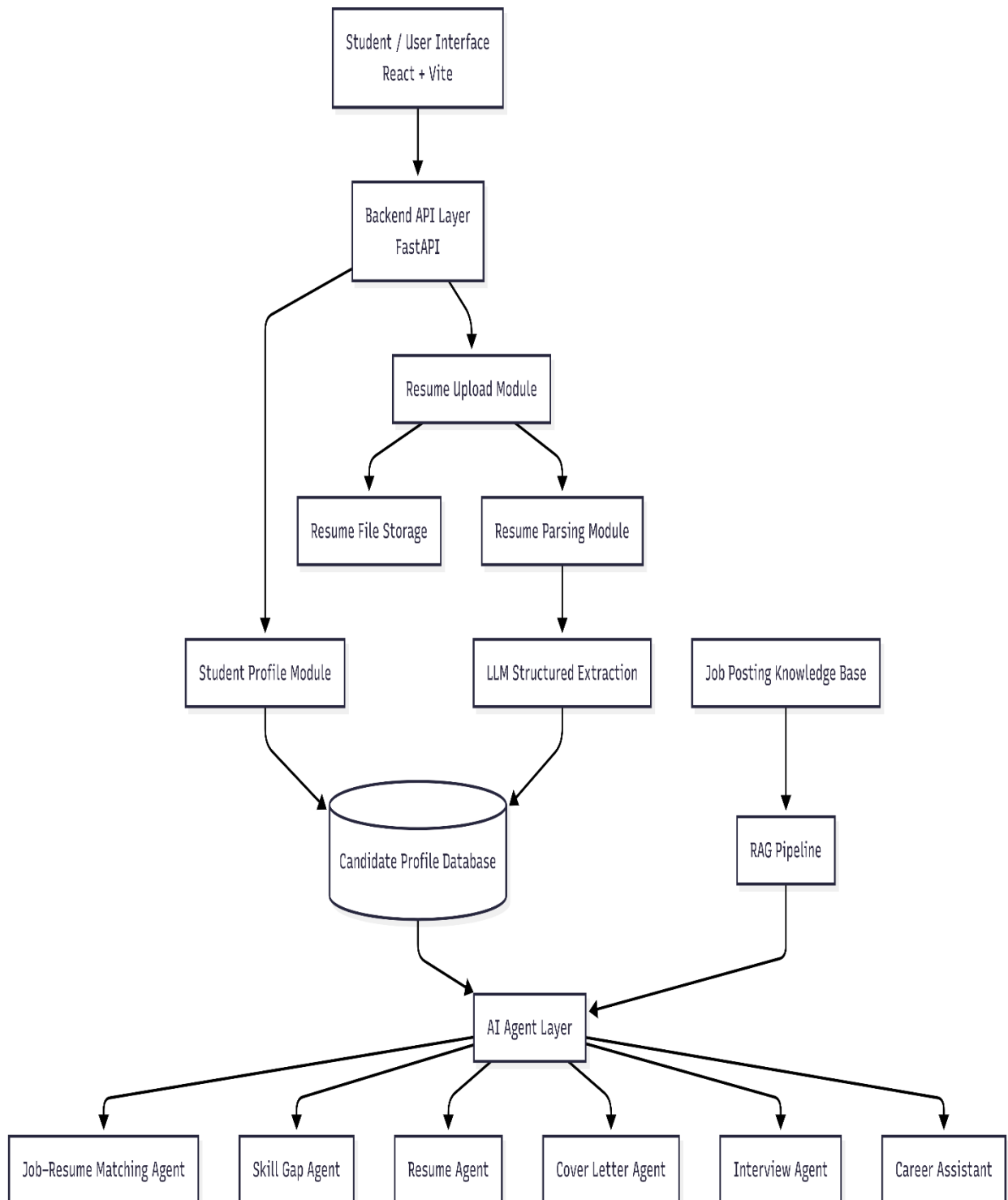
System Architecture and Tech Stack

Project id: #M-3-1

Submitted by: Bibi Fatima Yadwad

Submission Date: 28 August 2026

System Architecture and Tech Stack



AI Career Companion Agent for Internship Matching and Interview Preparation

1. Introduction

Applying for internships can be challenging for students because different internship opportunities have different eligibility criteria, technical requirements, preferred skills, and application expectations. Students often need to search for suitable opportunities, understand job descriptions, update their resumes, identify missing skills, prepare cover letters, practice interview questions, and track their applications. Managing all these activities manually can be time-consuming and may make it difficult for students to understand whether they are actually suitable for a particular internship.

The **AI Career Companion Agent** is proposed as an intelligent career-support system that assists students throughout the internship application process. The system will understand a student's profile and resume, extract important information such as skills, education, projects, certifications, and experience, and use this information to provide personalized career guidance.

The system is designed around a combination of **Large Language Models (LLMs)**, **Retrieval-Augmented Generation (RAG)**, and **specialized AI agents**. Instead of relying on a single AI component for every task, the proposed architecture divides career-related responsibilities among six specialized agents. This makes the system modular and allows individual components to be improved independently.

The first milestone focuses on establishing the foundation of the system. It includes understanding the internship application workflow, studying Retrieval-Augmented Generation, exploring multi-agent system design, selecting an appropriate technology stack, designing the system architecture, and implementing the student profile and resume-processing foundation.

The overall architecture follows the flow:

Student → React Frontend → FastAPI Backend → Profile + Resume Database → Resume Parser → LLM/Gemini → RAG Knowledge Base → Six Specialized Agents → Career Recommendations

2. Problem Statement

Students often face several difficulties while applying for internships:

- Finding internships that match their skills and interests.

- Understanding lengthy and technical job descriptions.
- Identifying whether they satisfy internship requirements.
- Determining which skills they are missing.
- Customizing their resume for different positions.
- Writing role-specific cover letters.
- Preparing for technical and behavioral interviews.
- Managing information related to multiple applications.
- Receiving personalized guidance rather than generic career advice.

Traditional job-search platforms mainly focus on displaying job opportunities and allowing students to apply. They do not necessarily provide an integrated AI-based system that understands the student's complete profile and continuously assists with matching, skill-gap analysis, resume improvement, cover-letter generation, and interview preparation.

The proposed AI Career Companion Agent aims to address this problem by creating a centralized intelligent system that combines structured student information with relevant internship and career knowledge.

3. Objectives

The main objectives of the AI Career Companion Agent are:

1. To create and maintain a structured student profile.
2. To allow students to upload their resumes in common formats such as PDF and DOCX.
3. To extract useful text and information from uploaded resumes.
4. To convert unstructured resume content into structured information.
5. To identify skills, education, experience, projects, certifications, and other relevant information.
6. To store profile and resume metadata in a database.
7. To use an LLM such as Gemini to assist with structured information extraction and career-related tasks.
8. To introduce a RAG-based knowledge system for internship and career information.

9. To design six specialized AI agents for different career-support tasks.
10. To provide personalized internship recommendations and career guidance.
11. To create a scalable architecture that can be extended in later milestones.

4. Internship Application Workflow

A typical internship application process consists of several stages.

Stage 1 – Student Profile Creation

The student provides basic information such as:

- Full name
- Email
- Phone number
- Location
- Target role
- LinkedIn profile

This information forms the initial candidate profile.

Stage 2 – Resume Upload

The student uploads a resume through the frontend. The backend receives the file and stores its metadata. The actual file can initially be stored locally during development.

Stage 3 – Resume Parsing

The system extracts text from the uploaded resume. PDF resumes can be processed using **PyPDF**, while DOCX resumes can be processed using **python-docx**.

Stage 4 – Structured Information Extraction

The extracted text is sent to the LLM/Gemini component. The model identifies important information such as:

- Professional summary
- Technical and soft skills

- Education
- Work experience
- Projects
- Certifications

The extracted information is stored as structured JSON so that it can be reused by other components of the system.

Stage 5 – Internship Matching

When internship information becomes available, the student's structured profile can be compared with internship requirements. The Job–Resume Matching Agent can determine how closely the student's skills and experience match a particular opportunity.

Stage 6 – Skill-Gap Analysis

The Skill Gap Agent identifies skills required by the internship that are missing or weak in the student's profile. It can then suggest learning priorities.

Stage 7 – Resume and Cover Letter Customization

The Resume Agent can suggest improvements to the student's resume for a selected role. The Cover Letter Agent can generate a personalized cover-letter draft based on the candidate profile and job description.

Stage 8 – Interview Preparation

The Interview Agent can generate questions relevant to the selected internship and evaluate the student's answers.

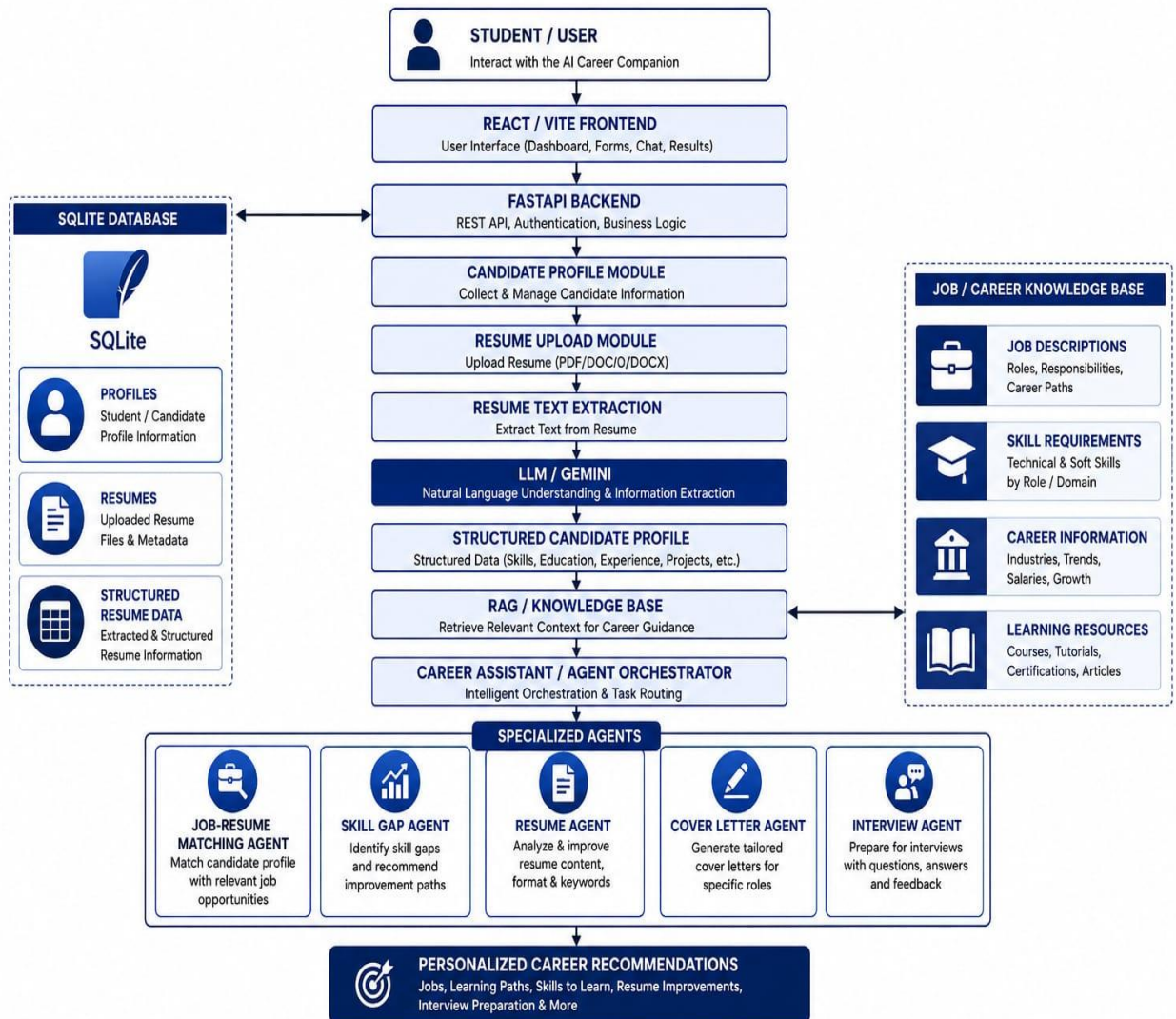
Stage 9 – Career Recommendations

The system combines the outputs of the relevant components and provides personalized recommendations to the student.

5. Proposed System Architecture

The proposed system follows a layered architecture in which the frontend, backend, database, AI processing components, knowledge base, and specialized agents work together.

5.1 High-Level Architecture



5.2 Explanation of the Architecture

Student

The student is the primary user of the system. The student interacts with the platform to create a profile, upload a resume, view extracted information, search or evaluate internships, and receive career recommendations.

React Frontend

The frontend is developed using **React with Vite**. It provides the user interface through which students can enter profile information, upload resumes, view extracted data, and interact with the AI career assistant.

The frontend communicates with the backend through REST APIs.

FastAPI Backend

FastAPI acts as the main backend layer. It handles:

- API requests
- Student profile creation
- Resume uploads
- Input validation
- Communication with the database
- Resume-processing requests
- Communication with AI services
- Returning structured responses to the frontend

The backend also acts as the central coordination layer between the frontend and the AI components.

Profile + Resume Database

The database stores student profile information and resume metadata. **SQLite** is selected for Milestone 1 because it is lightweight and easy to configure during development.

The architecture can later be migrated to PostgreSQL when the application requires larger-scale deployment and concurrent access.

Resume Parser

The resume parser converts uploaded resume files into machine-readable text.

- **PyPDF** is used for PDF resumes.
- **python-docx** is used for DOCX resumes.

The parser focuses on extracting the textual content while preserving enough structure for subsequent AI processing.

LLM / Gemini

The extracted resume text is processed using an LLM such as **Gemini**. The LLM identifies and organizes information from the resume into a structured representation.

For example, unstructured text such as:

Python, Java, SQL, TensorFlow

can be converted into a structured skills list.

Similarly, education, projects, certifications, and experience can be represented in structured JSON.

This structured profile becomes the foundation for later matching and recommendation tasks.

RAG Knowledge Base

The RAG layer will provide external and up-to-date information to the AI system. The knowledge base can contain:

- Internship job descriptions
- Job requirements
- Company information
- Skill requirements
- Interview resources
- Career preparation material

Instead of relying only on information stored in the LLM, the system retrieves relevant information from the knowledge base and supplies it as context to the model.

Six Specialized Agents

The system contains six specialized AI agents. Each agent has a clearly defined responsibility.

1. **Job–Resume Matching Agent**
Compares the candidate profile with internship requirements and provides a match score and explanation.
2. **Skill Gap Agent**
Identifies missing or weak skills and recommends learning priorities.
3. **Resume Agent**
Suggests improvements and role-specific modifications to the student's resume.
4. **Cover Letter Agent**
Generates a personalized cover-letter draft based on the student's profile and selected internship.

5. Interview Agent

Generates relevant interview questions and provides feedback on student answers.

6. Career Assistant

Handles general career questions and guides the student through the platform.

An orchestrator or backend service can determine which agent should handle a particular request.

Career Recommendations

The final output of the system consists of personalized recommendations. Depending on the student's request, the system may provide suitable internships, skill-gap information, resume suggestions, cover-letter content, interview questions, or general career guidance.

6. Retrieval-Augmented Generation (RAG)

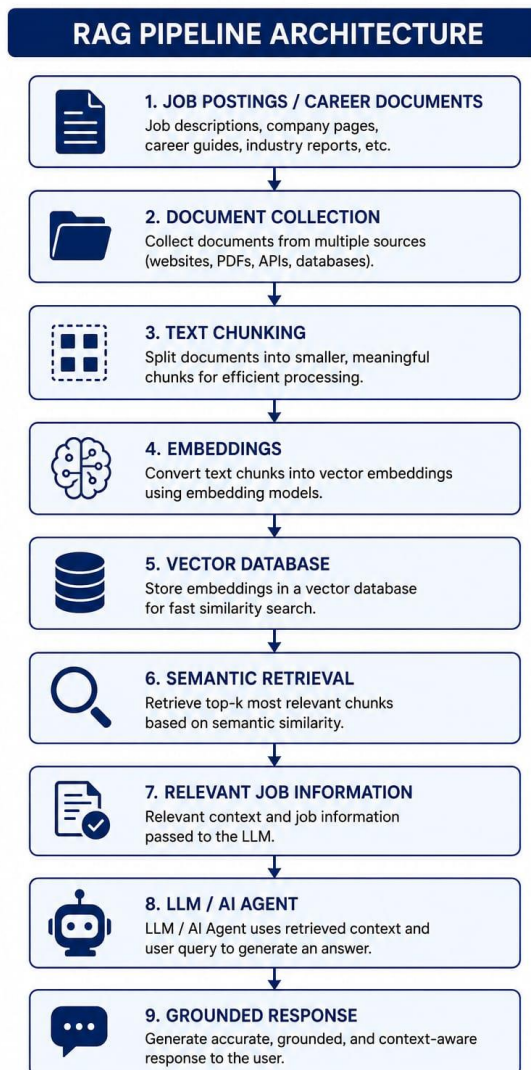
6.1 What is RAG?

Retrieval-Augmented Generation, commonly called **RAG**, is an AI technique that combines a Large Language Model with an external knowledge source.

A conventional LLM generates answers primarily from information learned during model training. However, internship information, company requirements, and job postings can change frequently. RAG addresses this problem by retrieving relevant information from an external knowledge base before generating a response.

For the AI Career Companion Agent, the RAG system will provide current and relevant job-related information to the AI agents.

6.2 RAG Workflow



Step 1 – Document Collection

Internship descriptions, job requirements, company information, and career resources are collected.

Step 2 – Document Chunking

Large documents are divided into smaller sections called chunks. Chunking makes it easier to retrieve only the relevant portions of a document.

Step 3 – Embeddings

Each chunk is converted into a numerical vector representation called an embedding.

Step 4 – Vector Storage

The embeddings are stored in a vector database such as **Chroma** or **pgvector**.

Step 5 – Retrieval

When a student asks a question, the system searches the vector database for information that is semantically similar to the query.

Step 6 – Context Injection

The retrieved information is provided to the LLM together with the student's structured profile.

Step 7 – Response Generation

The LLM or specialized agent uses the retrieved information to generate a grounded and personalized response.

For example, if a student asks:

"Am I suitable for this Python developer internship?"

the system can retrieve the internship requirements, compare them with the student's skills and experience, and explain:

- Matching skills
- Missing skills
- Relevant projects
- Eligibility concerns
- Recommended preparation

The original RAG research introduced the combination of a language model with retrieved information from an external knowledge source for knowledge-intensive tasks.

Reference: Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.

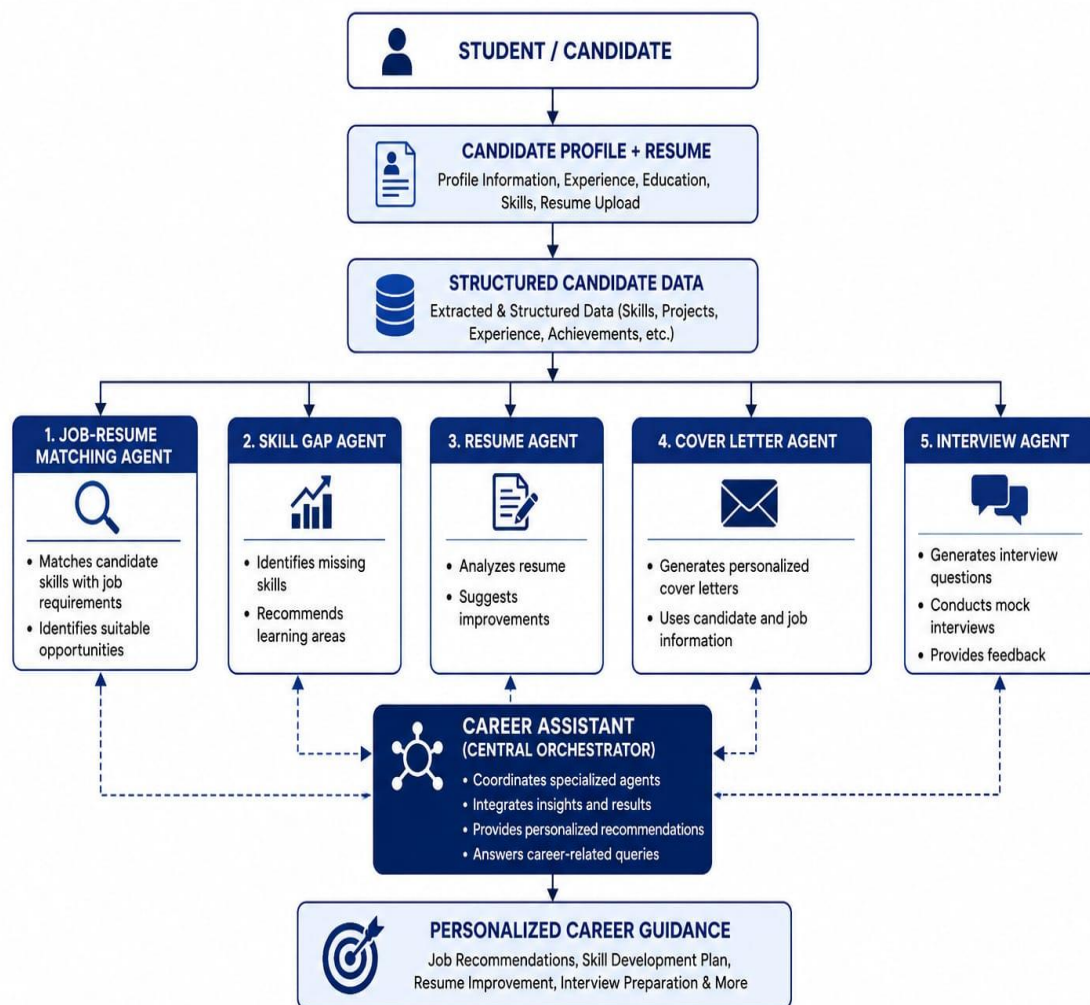
7. Multi-Agent System Design

7.1 Concept

A multi-agent system consists of multiple AI agents that perform different tasks. Instead of using one AI component for every career-related function, the proposed system divides the workload among specialized agents.

This approach provides modularity and allows each agent to focus on a specific task.

7.2 Proposed Agent Architecture



7.3 Job–Resume Matching Agent

This agent compares the student's structured profile with the requirements of a selected internship.

It can evaluate:

- Technical skills
- Education
- Projects
- Experience
- Certifications
- Role-specific requirements

The agent can produce a match score along with an explanation of why the student matches or does not match the position.

7.4 Skill Gap Agent

The Skill Gap Agent focuses on identifying missing or weak skills.

For example, if an internship requires:

- Python
- SQL
- Git
- REST APIs
- FastAPI

and the student has Python, SQL, and Git but lacks FastAPI and REST API experience, the agent can identify these as skill gaps and prioritize them.

7.5 Resume Agent

The Resume Agent analyzes the student's resume against a selected internship and suggests improvements.

It can recommend:

- Adding relevant skills
- Improving project descriptions
- Highlighting relevant experience
- Using stronger action-oriented descriptions
- Removing unnecessary information
- Better aligning the resume with the job description

The system should provide suggestions based on the student's actual experience rather than inventing qualifications.

7.6 Cover Letter Agent

The Cover Letter Agent creates a personalized cover-letter draft using:

- Student profile
- Resume information
- Target position
- Job description

- Relevant projects and skills

This reduces the need for students to write a completely new cover letter for every application.

7.7 Interview Agent

The Interview Agent assists with interview preparation.

It can generate:

- Technical questions
- Behavioral questions
- Project-based questions
- Role-specific questions
- Follow-up questions

It can also evaluate student answers and provide feedback on clarity, relevance, technical accuracy, and areas for improvement.

7.8 Career Assistant Agent

The Career Assistant acts as the general conversational component of the platform.

It can help students with questions such as:

- "Which skills should I learn next?"
- "How should I prepare for a Python internship?"
- "What projects should I highlight?"
- "How can I improve my interview preparation?"

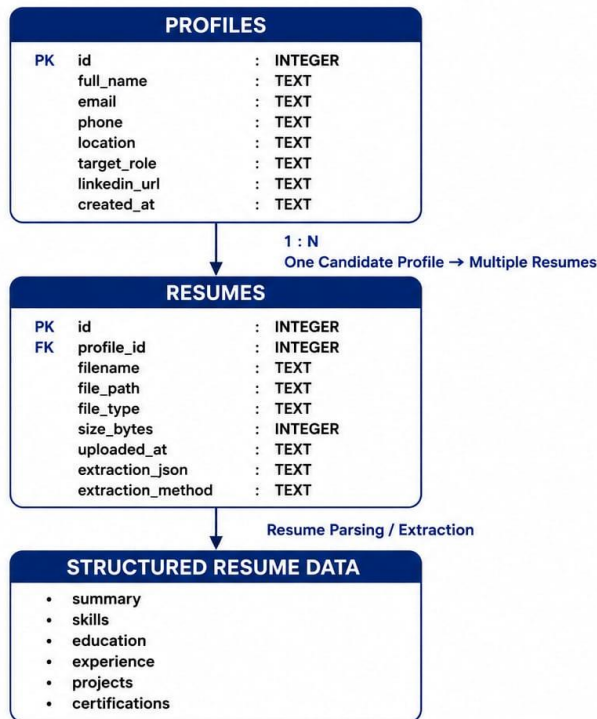
The Career Assistant can route specialized requests to the appropriate agent when necessary.

8. Database Design

The database is designed to maintain a relationship between student profiles and their uploaded resumes.

8.1 Entity Relationship Overview

Candidate Profile & Resume Database Schema



8.2 Profiles Table

The **Profiles** table stores the student's basic information.

Field	Description
id	Primary key identifying the profile
full_name	Student's full name
email	Student's email address
phone	Student's phone number
location	Student's current/preferred location
target_role	Internship or career role targeted by the student
linkedin_url	LinkedIn profile URL
created_at	Date and time when the profile was created

The profile acts as the main entity for connecting student information with uploaded resumes.

8.3 Resumes Table

The **Resumes** table stores information about each uploaded resume.

Field	Description
id	Primary key identifying the resume
profile_id	Foreign key connecting the resume to a student profile
filename	Original uploaded filename
file_path	Location where the file is stored
file_type	File format such as PDF or DOCX
size_bytes	Size of the uploaded file
uploaded_at	Date and time of upload
extraction_json	Structured information extracted from the resume
extraction_method	Method/model used for extraction

A single student can upload multiple resumes. Therefore, the relationship between **Profiles** and **Resumes** is **one-to-many (1:N)**.

8.4 Structured Resume

The extracted information is represented using structured fields such as:

Summary

A concise description of the student's professional profile.

Skills

Technical and soft skills identified from the resume.

Education

Details such as:

- Degree
- Institution
- Graduation year
- Academic performance

Experience

Information about internships, jobs, or other relevant professional experience.

Projects

Project names, descriptions, technologies, and relevant contributions.

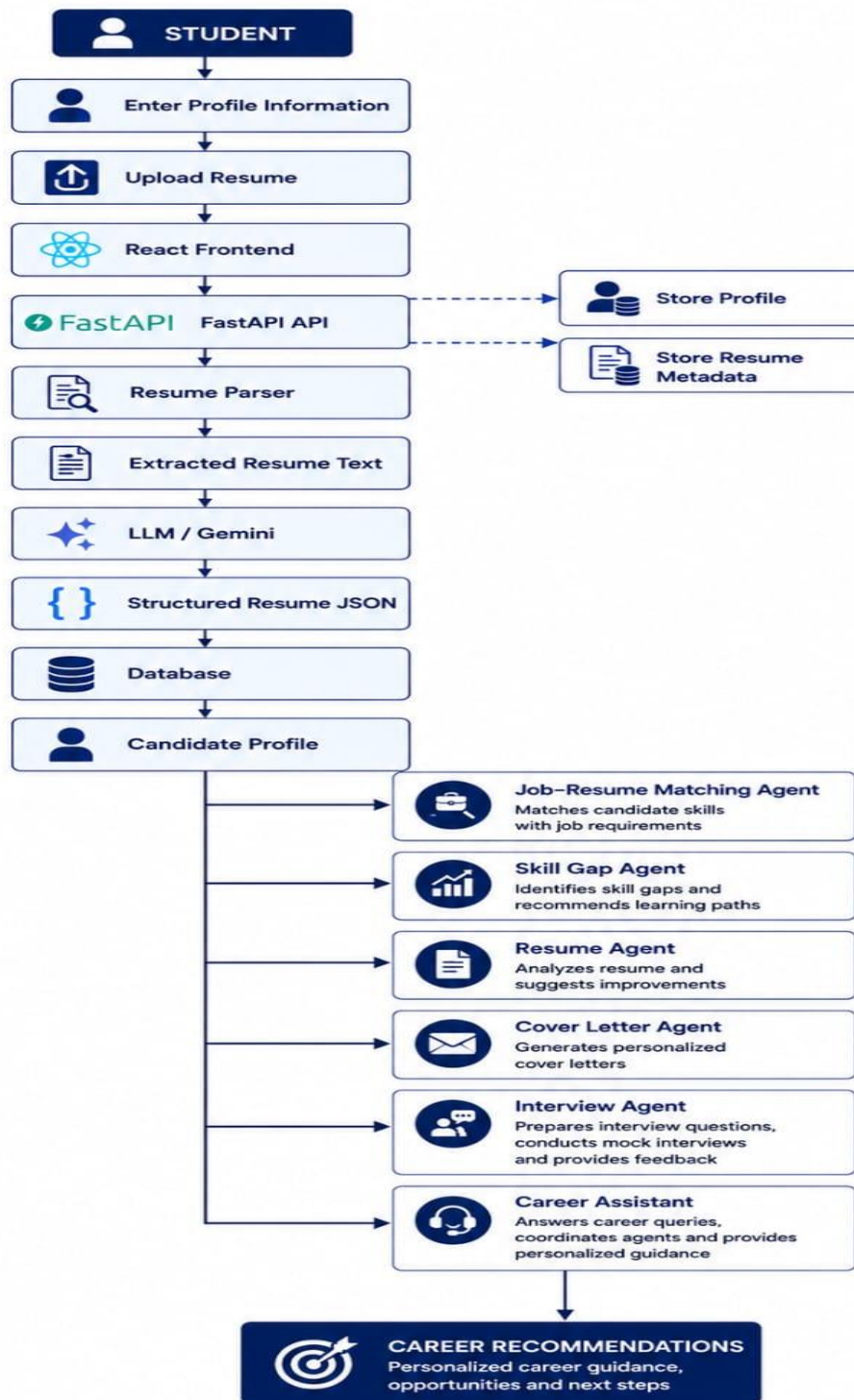
Certifications

Relevant certifications and completed courses.

The structured representation allows the AI agents to access consistent information rather than repeatedly processing the original resume.

9. Data Flow from Resume Upload to AI Recommendations

The complete data flow can be represented as follows:



For tasks involving external internship information, the agents can additionally retrieve relevant information through the RAG knowledge base.

10. Technology Stack

Layer	Technology	Reason for Selection
-------	------------	----------------------

Frontend	React + Vite	Provides a fast and modular interface for students
Backend	FastAPI	Supports REST APIs, validation, file uploads, and AI integration
Database	SQLite	Lightweight and simple for Milestone 1 development
Future Database	PostgreSQL	Suitable for scalable production deployment
Resume Storage	Local File Storage	Simple solution for development and testing
PDF Parsing	PyPDF	Extracts text from PDF resumes
DOCX Parsing	python-docx	Extracts text from DOCX resumes
LLM	Gemini	Supports natural-language understanding and structured extraction
RAG Framework/Store	Chroma or pgvector	Enables semantic retrieval from career knowledge
AI Agents	LLM-based specialized agents	Separates career tasks into modular components
API Communication	REST APIs	Connects frontend and backend components

11. Why This Architecture is Suitable

The proposed architecture provides several advantages.

Modularity

Each component has a specific responsibility. The resume parser, LLM extraction component, RAG system, and specialized agents can be developed and improved independently.

Reusability

Once a resume has been converted into structured information, multiple agents can reuse the same candidate profile.

Personalization

The system uses student-specific information rather than providing only generic career advice.

Extensibility

Additional agents can be introduced later without redesigning the complete system.

Scalability

SQLite is sufficient for initial development, while PostgreSQL and pgvector can be introduced later for larger deployments.

Grounded AI Responses

The RAG component allows the system to retrieve relevant job information instead of depending only on the LLM's general knowledge.

12. Milestone 1 Scope

Milestone 1 focuses on establishing the candidate-profile and resume-processing foundation.

The following components are included in the initial implementation:

Implemented/Planned for Milestone 1

- Student profile creation
- Profile information storage
- Resume upload
- Resume metadata storage
- Local resume file storage
- PDF resume parsing
- DOCX resume parsing
- Resume text extraction
- LLM/Gemini-based structured extraction
- Extraction of skills
- Extraction of education

- Extraction of experience
- Extraction of projects
- Extraction of certifications
- Structured resume JSON storage
- Initial system architecture
- Database schema design

Future Milestones

The following components are part of the overall architecture but can be implemented progressively after the candidate-profile foundation is stable:

- Internship/job data collection
- RAG knowledge base
- Vector database integration
- Job–Resume Matching Agent
- Skill Gap Agent
- Resume Agent
- Cover Letter Agent
- Interview Agent
- Career Assistant Agent
- Agent orchestration
- Application tracking
- Advanced recommendation system
- Production database and deployment

This staged implementation reduces complexity and allows the core data pipeline to be tested before introducing multiple AI agents.

13. Expected System Workflow

After completing the planned architecture, the expected end-to-end workflow will be:

1. The student creates an account/profile.
2. The student enters basic career information.
3. The student uploads a resume.
4. The React frontend sends the information to the FastAPI backend.
5. FastAPI validates the input and stores profile and resume metadata.
6. The resume parser extracts text from the uploaded file.
7. Gemini processes the extracted text.
8. The system generates structured resume information.
9. The structured information is stored in the database.
10. When the student selects an internship, the system retrieves relevant job information through RAG.
11. The appropriate specialized agent analyzes the student and internship information.
12. The system generates personalized recommendations.
13. The recommendations are displayed through the React frontend.

14. Example Use Case

Consider a student applying for a **Software Engineering Internship**.

The student uploads a resume containing Python, Java, SQL, machine-learning projects, and a few academic projects.

The system first extracts the resume information and creates a structured candidate profile.

Suppose the internship requires:

- Python
- Git
- REST APIs
- FastAPI
- SQL
- Cloud fundamentals

The Job–Resume Matching Agent can identify that the student already has Python, Git, and SQL-related experience.

The Skill Gap Agent can identify REST APIs, FastAPI, and cloud fundamentals as areas requiring additional preparation.

The Resume Agent can recommend highlighting relevant Python and backend projects.

The Cover Letter Agent can create a role-specific cover-letter draft.

The Interview Agent can generate questions related to Python, REST APIs, SQL, and the student's projects.

Finally, the Career Assistant can summarize the student's overall suitability and recommend the most important preparation steps.

This demonstrates how the different components work together to provide a personalized career-support experience.

15. Future Enhancements

After the completion of the initial milestone, the system can be expanded with additional capabilities.

Automated Internship Discovery

The system can collect internship opportunities from approved job sources and organize them based on role, location, skills, and eligibility.

Advanced Matching

Matching can be improved using semantic similarity between student profiles and job descriptions.

Personalized Learning Recommendations

The Skill Gap Agent can recommend courses, tutorials, projects, and practice resources for missing skills.

Interview Simulation

The Interview Agent can support interactive mock interviews where students answer questions and receive AI-generated feedback.

Application Tracking

Students can maintain application statuses such as:

- Saved
- Applied
- Assessment
- Interview
- Selected
- Rejected

Production Database

SQLite can be replaced with PostgreSQL for a production environment.

Vector Database

Chroma or pgvector can be integrated to support semantic search over job descriptions and career resources.

Improved Agent Orchestration

A dedicated orchestrator can dynamically determine which agents should participate in a complex request and combine their results.

16. Conclusion

The proposed **AI Career Companion Agent** provides an integrated approach to internship preparation by combining structured student profiles, resume processing, Large Language Models, Retrieval-Augmented Generation, and specialized AI agents.

The foundation of the system is the student profile and structured resume. The React frontend provides an accessible interface, while FastAPI manages API communication, validation, file uploads, and coordination with the AI components. SQLite provides simple and reliable storage during the initial milestone.

The resume parser converts PDF and DOCX resumes into text, after which Gemini can transform the unstructured content into structured information such as skills, education, experience, projects, and certifications. This structured candidate profile can then be reused by multiple AI agents.

The future RAG layer will allow the system to retrieve relevant internship and career information from an external knowledge base. This will make AI

responses more relevant and grounded in the specific job information being analyzed.

The six specialized agents—**Job–Resume Matching Agent, Skill Gap Agent, Resume Agent, Cover Letter Agent, Interview Agent, and Career Assistant**—divide the career-support process into manageable tasks. This modular architecture makes the system easier to develop, maintain, test, and extend.

For Milestone 1, the primary focus is therefore on building a reliable candidate-profile and resume-processing pipeline. Once this foundation is stable, the RAG pipeline, multi-agent system, internship matching, interview preparation, and personalized career recommendations can be progressively integrated.

Overall, the proposed architecture provides a practical foundation for developing an intelligent career companion that can guide students from **resume preparation and internship matching to skill development and interview preparation** within a single platform.